

O que são testes de integração?

Testes de integração são uma etapa do processo de testes de software focada em verificar se **diferentes partes do sistema funcionam corretamente quando combinadas**. Enquanto os testes unitários validam pequenas partes isoladas (como métodos ou classes), os testes de integração validam **a comunicação entre esses componentes**.

São testes que verificam a interação entre módulos, como por exemplo:

- Camada de serviço ↔ banco de dados
- API ↔ camada de negócio
- Sistema ↔ serviços externos (APIs, filas, etc.)

👉 Ou seja, eles garantem que a “**conversa**” entre as partes funciona corretamente.

Objetivos principais

- Detectar erros de comunicação entre módulos
- Verificar integração com banco de dados
- Validar fluxos completos (quase reais)
- Garantir que dependências externas funcionem corretamente

Conexão com banco (H2)

```
import java.sql.Connection;
import java.sql.DriverManager;

public class Conexao {

    private static Connection conn;

    public static Connection getConnection() throws Exception {
        if (conn == null || conn.isClosed()) {
            conn = DriverManager.getConnection(
                "jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1", "sa", ""
            );
        }
        return conn;
    }
}
```

DAO (acesso ao banco)

```
import java.sql.*;

public class ContaDAO {

    public void criarTabela() throws Exception {
        Connection conn = Conexao.getConnection();
        Statement st = conn.createStatement();

        st.execute("CREATE TABLE conta (id INT AUTO_INCREMENT
PRIMARY KEY, titular VARCHAR(100), saldo DOUBLE)");
        conn.close();
    }

    public void salvar(Conta conta) throws Exception {
        Connection conn = Conexao.getConnection();

        PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO conta (titular, saldo) VALUES (?, ?)"
        );

        ps.setString(1, conta.getTitular());
        ps.setDouble(2, conta.getSaldo());

        ps.executeUpdate();
        conn.close();
    }

    public int contarRegistros() throws Exception {
        Connection conn = Conexao.getConnection();
        Statement st = conn.createStatement();

        ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM
conta");
        rs.next();
    }
}
```

```
        int total = rs.getInt(1);
        conn.close();

        return total;
    }
}
```

Teste de Integração

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ContaDAOTest {

    @Test
    public void deveInserirContaNoBanco() throws Exception {

        ContaDAO dao = new ContaDAO();

        dao.criarTabela(); // prepara banco

        Conta conta = new Conta("João", 1000.0);

        dao.salvar(conta);

        int total = dao.contarRegistros();

        assertEquals(1, total);
    }
}
```

Dependências necessárias

1. JUnit (para testes)
2. H2 Database (banco em memória)

pom.xml completo

```
<project
xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.exemplo</groupId>
    <artifactId>teste-integracao-simples</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>

        <!-- JUnit 5 -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.2</version>
            <scope>test</scope>
        </dependency>

        <!-- Banco H2 -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <version>2.2.224</version>
        </dependency>

    </dependencies>

    <build>
        <plugins>
```

```
        <!-- Plugin para rodar testes -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>

    </plugins>
</build>

</project>
```

O que cada dependência faz

JUnit

- Executa os testes
- Permite usar @Test, assertEquals, etc.

H2

- Banco de dados em memória
- Não precisa instalar nada
- Perfeito pra aula

Surefire Plugin

- Faz o Maven rodar os testes .

Dica importante

Se quiser que o banco **não suma ao finalizar**, pode mudar a conexão:

```
jdbc:h2:file:./meubanco
```

TESTE: transferir entre contas

Vamos fazer em 2 partes:

1.  Criar o teste primeiro (TDD)
2.  Implementar o método no DAO

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ContaDAOTest {

    @Test
    public void deveTransferirValorEntreContas() throws Exception
    {

        ContaDAO dao = new ContaDAO();
        dao.criarTabela();

        // cria contas
        Conta origem = new Conta("João", 1000.0);
        Conta destino = new Conta("Maria", 500.0);

        dao.salvar(origem);
        dao.salvar(destino);

        // realiza transferência
        dao.transferir(1, 2, 200.0);

        // verifica resultados
        double saldoOrigem = dao.buscarSaldo(1);
        double saldoDestino = dao.buscarSaldo(2);

        assertEquals(800.0, saldoOrigem);
        assertEquals(700.0, saldoDestino);
    }
}
```

Métodos auxiliares no DAO

Antes do transferir, precisamos de um método para buscar saldo:

```
public double buscarSaldo(int id) throws Exception {
```

```

Connection conn = Conexao.getConnection();

PreparedStatement ps = conn.prepareStatement(
    "SELECT saldo FROM conta WHERE id = ?"
);

ps.setInt(1, id);

ResultSet rs = ps.executeQuery();

if (rs.next()) {
    return rs.getDouble("saldo");
}

throw new RuntimeException("Conta não encontrada");
}

```

Método transferir no DAO

```

public void transferir(int origemId, int destinoId, double valor)
throws Exception {

    Connection conn = Conexao.getConnection();

    try {
        conn.setAutoCommit(false); // inicia transação

        // verifica saldo origem
        double saldoOrigem = buscarSaldo(origemId);

        if (saldoOrigem < valor) {
            throw new RuntimeException("Saldo insuficiente");
        }

        // debita origem
        PreparedStatement ps1 = conn.prepareStatement(
            "UPDATE conta SET saldo = saldo - ? WHERE id = ?"
        );
        ps1.setDouble(1, valor);
    }
}

```

```
ps1.setInt(2, origemId);
ps1.executeUpdate();

// credita destino
PreparedStatement ps2 = conn.prepareStatement(
    "UPDATE conta SET saldo = saldo + ? WHERE id = ?"
);
ps2.setDouble(1, valor);
ps2.setInt(2, destinoId);
ps2.executeUpdate();

conn.commit(); // confirma

} catch (Exception e) {
    conn.rollback(); // desfaz tudo
    throw e;
} finally {
    conn.setAutoCommit(true);
}
}
```